QUANT.OS

Institutional Algorithmic Trading, Multi-Asset Market Data & Risk Platform

v2.0.0 ENTERPRISE

Architecture & Operations Manual

TABLE OF CONTENTS

1. System Technology Stack & Architecture Overview

Quant.OS is built on a decoupled, multi-process architecture engineered for high throughput, sub-10ms tick ingestion, deterministic quantitative analysis, and zero cross-asset contamination.

LAYER	TECHNOLOGIES	RESPONSIBILITIES
Frontend	Next.js 14, React 18, TypeScript, Tailwind CSS, Zustand, React Query	Institutional Trading Desk, Pro Charting, Stock Screener, Risk Terminal, Bot Fleet Management
Backend API	Python 3.10+, Flask, SQLite3, Multi-threading, Pytest	REST API, SSE Event Streams, Strategy Evaluation, Technical Pipelines, Portfolio Accounting
Gateway	AsyncIO, WebSockets, Protobuf, Upstox SDK, CCXT, Delta API	High-frequency market feeds, Tick multiplexing, Failover recovery, Decimal normalization
Persistence	SQLite (`trading_platform.db`), Thread-Safe TTL Caches	Instrument Catalogs, Bot Configuration, Immutable Trade Ledgers, User Presets & Favorites

2. Complete Master Directory & File Map

Detailed file locations and responsibilities across the entire workspace:

2.1 Python Backend Core (`src/`)

FILE PATH	DESCRIPTION & KEY FUNCTIONS
`src/config.py`	Centralized configuration, environment variables loader, and exchange URL registries.
`src/db.py`	SQLite database engine, connection pooling, transactional queries, and self-healing schema migrations.
`src/bot_runtime_service.py`	Bot lifecycle controller (spawn, start, pause, stop, parameter hot-reloading).
`src/live_runner.py`	Real-time algorithmic execution engine evaluating strategy signals against incoming tick streams.
`src/universal_risk_engine.py`	Global risk firewall: max drawdown circuit breaker, position sizing limits, and kill-switch control.
`src/trade_ledger.py`	Immutable transaction ledger recording fills, slippage, realized P&L, and execution timestamps.
`src/indicators.py`	50+ mathematical indicator calculations: RSI, EMA (9/20/50/200), MACD, ATR, Bollinger, SuperTrend.
`src/option_chain_engine.py`	Options pricing engine calculating Black-Scholes Greeks (Delta, Gamma, Theta, Vega, IV) and strike ladders.
`src/upstox_service.py`	Upstox broker integration for Indian Equities (NSE/BSE) and Derivatives (NIFTY/BANKNIFTY).
`src/delta_options_service.py`	Delta Exchange integration for BTC/ETH crypto options and perpetual contracts.
`src/binance_market_data_service.py`	Binance real-time spot and USDT-M futures tick processor.

2.2 Modular Market Data Layer (`market_data/`)

FILE PATH	ROLE IN STOCKS & MULTI-ASSET ARCHITECTURE
`market_data/common/provider_interfaces.py`	Abstract base classes for market providers, streaming contracts, and capability bitmasks.
`market_data/common/canonical_ids.py`	Unified instrument identifier parser & resolver (`{provider}:{exchange}:{key}`).
`market_data/stocks/taxonomy.py`	Pure equity classification engine ensuring 0 derivative options leaks on stocks or spot crypto.
`market_data/stocks/quote_engine.py`	Real-time snapshot engine producing normalized stock quotes with Decimal arithmetic.
`market_data/stocks/analysis_engine.py`	Deterministic quantitative scoring engine (0-100 score, directional bias, plain-English summary).
`market_data/stocks/fundamentals_engine.py`	Corporate filings multiples (P/E, P/B, EPS, ROE, Debt/Equity) with strict `None` fallback.
`market_data/stocks/technical_engine.py`	Technical indicators & floor trader pivot levels (R1, R2, PP, S1, S2).
`market_data/stocks/screener_engine.py`	Multi-parameter filtered, sorted, and paginated screener engine.
`market_data/stocks/routes.py`	Flask blueprint registering all `/api/market-data/stocks` REST & SSE routes.

2.3 Frontend Application (`frontend/`)

DIRECTORY / COMPONENT	DESCRIPTION
`frontend/app/markets/page.tsx`	Unified Markets & Screener page hosting the dedicated Stocks Universe and Multi-Asset tables.
`frontend/src/features/markets/stocks/`	Self-contained Stocks Universe feature (API client, Zustand store, React Query hooks, and components).
`StocksUniverseView.tsx`	Master Stocks Screener container orchestrating headers, search chips, table, and details drawer.
`StockDetailsDrawer.tsx`	6-tab sliding drawer: Overview (OHLC/52W), SVG Chart, Analysis, Fundamentals, Technicals, Diagnostics.
`StockFiltersDrawer.tsx`	10-category collapsible filter drawer with instant match preview and reset actions.
`frontend/app/dashboard/page.tsx`	Command Center overview displaying fleet KPIs, active positions, P&L charts, and system status.
`frontend/app/charts/page.tsx`	TradingView Pro multi-timeframe interactive charting workstation.
`frontend/app/bots/page.tsx`	Bot Fleet Commander: spawn new bots, monitor decision logs, view real-time tick execution.
`frontend/app/options/page.tsx`	Options Command Center featuring live strike ladders, Greeks matrix, and multi-leg strategy builder.
`frontend/app/risk/page.tsx`	Universal Risk Engine forensics, max loss monitors, and emergency kill-switch controls.

3. Database Schemas & Models

```
-- 1. Master Instruments Catalog (trading_platform.db)
CREATE TABLE instruments (
    instrument_id TEXT PRIMARY KEY,       -- e.g. "upstox:NSE:RELIANCE"
    symbol TEXT NOT NULL,                 -- e.g. "RELIANCE"
    company_name TEXT,                    -- e.g. "Reliance Industries Limited"
    asset_class TEXT NOT NULL,            -- STOCKS, CRYPTO, FUTURES, OPTIONS, FOREX
    exchange TEXT NOT NULL,               -- NSE, BSE, NASDAQ, NYSE, BINANCE, DELTA
    currency TEXT DEFAULT 'INR',          -- INR, USD, USDT
    lot_size INTEGER DEFAULT 1,
    tick_size REAL DEFAULT 0.05,
    isin TEXT,
    is_fno_enabled BOOLEAN DEFAULT 0
);

-- 2. Algorithmic Bot Instances
CREATE TABLE bots (
    bot_id TEXT PRIMARY KEY,              -- e.g. "bot_a8f9c1"
    name TEXT NOT NULL,
    strategy_name TEXT NOT NULL,          -- e.g. "Momentum_Confluence_v1"
    symbol TEXT NOT NULL,
    status TEXT DEFAULT 'ACTIVE',         -- ACTIVE, PAUSED, HALTED, STOPPED
    allocated_capital REAL NOT NULL,
    max_drawdown_pct REAL DEFAULT 5.0,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 3. Immutable Trade & Order Ledger
CREATE TABLE trades (
    trade_id TEXT PRIMARY KEY,
    bot_id TEXT,
    symbol TEXT NOT NULL,
    side TEXT NOT NULL,                   -- BUY, SELL
    quantity REAL NOT NULL,
    price REAL NOT NULL,
    pnl REAL DEFAULT 0.0,
    status TEXT DEFAULT 'FILLED',
    timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

4. Standard API Envelope & Core Endpoints

```
// Standard API Envelope Contract (ApiResponseEnvelope)
{
  "status": "success" | "error",
  "data": { ... },
  "meta": {
    "total": 19,
    "page": 1,
    "page_size": 50,
    "receivedTimestamp": 1788090000000,
    "latency_ms": 4.2
  },
  "errors": []
}
```

ENDPOINT	METHOD	DESCRIPTION
`/api/market-data/stocks`	`GET`	Paginated pure equity screener with multi-criteria filters and sorting.
`/api/market-data/stocks/<id>`	`GET`	Real-time stock quote snapshot with OHLC, 52W range, and volume.
`/api/market-data/stocks/<id>/history`	`GET`	Multi-timeframe OHLCV historical candle series (`1m` to `1d`).
`/api/market-data/stocks/<id>/analysis`	`GET`	Explainable quantitative analysis (0-100 score & English reasoning).
`/api/market-data/stocks/movers`	`GET`	Ranked Top Gainers, Losers, and Most Active equities.
`/api/bots`	`GET/POST`	List active trading bots or spawn new algorithmic bot instances.
`/api/risk/status`	`GET`	Universal risk status, drawdown meters, and global kill-switch state.

5. Operations & Management Guide

5.1 Launching the Complete System

```
# Single-command launch from repository root:
python run_system.py

# Or launch services independently:
python dashboard.py          # Port 5050 (Flask Backend API)
python start_gateway.py      # Port 5051 (Market Data Gateway)
cd frontend && npm run dev   # Port 3100 (Next.js Frontend)
```

5.2 Verification & Automated Testing

```
# 1. Run Python Backend Test Suite (Pytest)
python -m pytest market_data/stocks/tests/

# 2. Run TypeScript Typecheck
cd frontend && npm run typecheck

# 3. Run Automated Chrome Browser Verification
node scripts/test_stocks_universe_browser.js
```

5.3 Emergency Controls

- **Global Emergency Kill-Switch:** Click the red `HALT` button on the top command bar or write `{"kill": true}` to `kill_switch.flag`. All active bots will immediately pause and cancel unfulfilled orders.
- **Database Reset & Cleanup:** Run `python clean_leftover_test_bots.py` to clear orphaned test bots.